大和セキュリティ & KSEC presents　セキュリティツール Vibe コーディング入門＆大会

AIと作り、AIが使う セキュリティツール

インターフェース・型・テスト・実行権限

一般社団法人鹿児島県サイバーセキュリティ協議会 代表理事 西川 彰

自己紹介

西川 彰（Akira Nishikawa）

鹿児島県サイバーセキュリティ協議会

代表理事

株式会社サイバーセキュリティクラウド

AWS Community Builder

CISSP

主な活動

・県内の勉強会・セミナーを企画運営

・病院のセキュリティコンサルタント

・開発しているOSS：

- Hayabusa (最初だけ)
- Takajo
- Senrigan(ちょっとだけ)

協議会は2017年に発足。県内の「知る・守る・続ける」を支える非営利団体

ツールを操作する相手に、AIが加わる

人間が直接使う設計

覚えやすいコマンド

少ない操作で目的を達成

人が読んで判断する出力

AIを介して使う設計

AIが引数の選択を支援

細かな分析機能も公開できる

結果を次の処理へつなぐ

人が全オプションを覚えて使う、という制約を緩められる

高機能化を支える、明確なインターフェース

人が覚えきれない機能も、AIが扱える設計へ

引数の意味・型・組み合わせを明示

JSONなどの構造化出力と、一貫したエラー

調査・プレビューと、変更の実行を分離

複雑でもドキュメントさえあれで AI は使いこなすことができる

AIに任せる前の「合格条件」

何ができたら完成かを先に決める

例：ログ解析ツール

既知の攻撃ログを検出できる

正常ログを誤検知しない

壊れた行があっても、処理結果とエラーが分かる

より具体的であればあるほど AI はそれを満たそうとしてくれる

言語は静的型付けのものを強く推奨

型で表せる制約は型検査に任せる

Go / Rust / TypeScript など

引数・戻り値・データ構造の不整合を早く見つける

型エラーをAIに返して、修正を繰り返すことができる

何も理由がなければ Rust がよい

今後はメモリ安全性がより重要になる（はず）

既存のコード・ライブラリ・チームの習熟度も判断材料

型が減らせるテスト、残るテスト

型検査に任せられる例

数値の引数に文字列を渡す

存在しないフィールドを参照する

戻り値の型を取り違える

動作として確かめる例

検出条件は正しいか

境界値や空データでどう動くか

他人のデータを取得できないか

削減できるのは「型で保証した性質」の確認。削減量は設計次第

テストは「失敗してほしい条件」まで

正常系だけでは守りたい性質が抜ける

ログ解析なら、検出対象と非対象の両方

境界値・欠損・壊れた入力

修正した不具合を、再現テストとして残す

認可のある機能なら「権限のない操作を拒否する」も確認

AIが書いたテストもレビュー対象

実装とテストが同じ誤解を持ちうる

期待値は、仕様や既知の正解から決める

失敗時は、実装とテストのどちらが違うか確認

バグ修正では、修正前に失敗し、修正後に通ることを確認

ガードレールは仕組みを分けて考える

モデルの判断に依存

「本番を変更しないで」

「秘密情報を送信しないで」

検知モデルによる危険判定

CLAUDE.md だけではどうにもならない

（保証もないのでお気持ち表明）

実行環境で強制

- 本番の資格情報を渡さない
- 書き込み可能な範囲を制限
- 通信先・ツール権限を制限
  - Hooks や別のレイヤーで制限

文章の指示だけで厳密な制約を保証しない。強制の範囲・例外も確認

AIが間違えても、影響を限定する

作業に必要な権限だけを渡す

検証用の環境とデータを使う

本番への反映・削除は、独立した承認経路へ

操作記録を残し、復元方法を用意する

コンテナでも、ホストのマウント・資格情報・外向き通信を確認

小さい変更を、自動検証して取り込む

01　入力例と期待する結果を決める

02　小さな単位で実装を依頼する

03　型検査とテストをCIで実行する

04　人が差分と失敗時の影響を確認する

05.　（とはいえ自由にやってよしgoal で一直線も全然あり

```
/goal all tests in test/auth pass and the lint step is clean
```

CI設定・テスト・権限設定の変更もレビュー対象

個人のOSSは、好奇心から始めていい

「この技術を使いたい！」
「それなら、何が作れそう？」

小さく作って、自分で使ってみる
役立つ場面が見つかったら、用途や品質を磨く
一人の開発はスクラップ＆ビルドで全然 OK！気軽にやりましょう！

非機能要件に差が出る！

セキュリティガードが働いたら

- モデルを変えてみる
- タスクを細かく区切る
- 該当部分だけ気合いで自分で実装する

引っかかった部分だけ別のタスクや別の LLM に作成させるでも OK。作るのは断ってくるが、すでにあるコードに文句を言うことはなかなかない

今年作ったツールやシステム

- ハニーポット（Deception 環境）
- フィッシングサイト通報システム
- 脆弱性情報収集＆トリアージシステム
- Web アクセスログの脅威ハンティングツール
- VPN ログイン通知システム
- EPSS チェッカー
- …etc

とにかく手を動かして経験を積みましょう！！

おすすめスライド



https://speakerdeck.com/prog893/ai-coding-and-human-cognitive-throughput